WHITE BOX TESTING – MoneyPoly

1.1 Control Flow Graph

The Control Flow Graph (CFG) was constructed for the core function play_turn() in the MoneyPoly game. This function governs the execution of a player's turn, including dice rolling, jail handling, movement, and turn advancement.

Description

The execution begins by selecting the current player. A check is performed to determine whether the player is in jail. If the player is in jail, jail handling logic is executed, and the turn ends.

If the player is not in jail, the dice are rolled. If the player rolls doubles three consecutive times, they are sent to jail and the turn ends. Otherwise, the player moves across the board, and the tile on which they land is processed.

If the dice roll results in doubles, the player gets an additional turn. Otherwise, the turn advances to the next player.

Cyclomatic Complexity

Cyclomatic Complexity is calculated using:

M = Number of decision points + 1

Decision points identified:

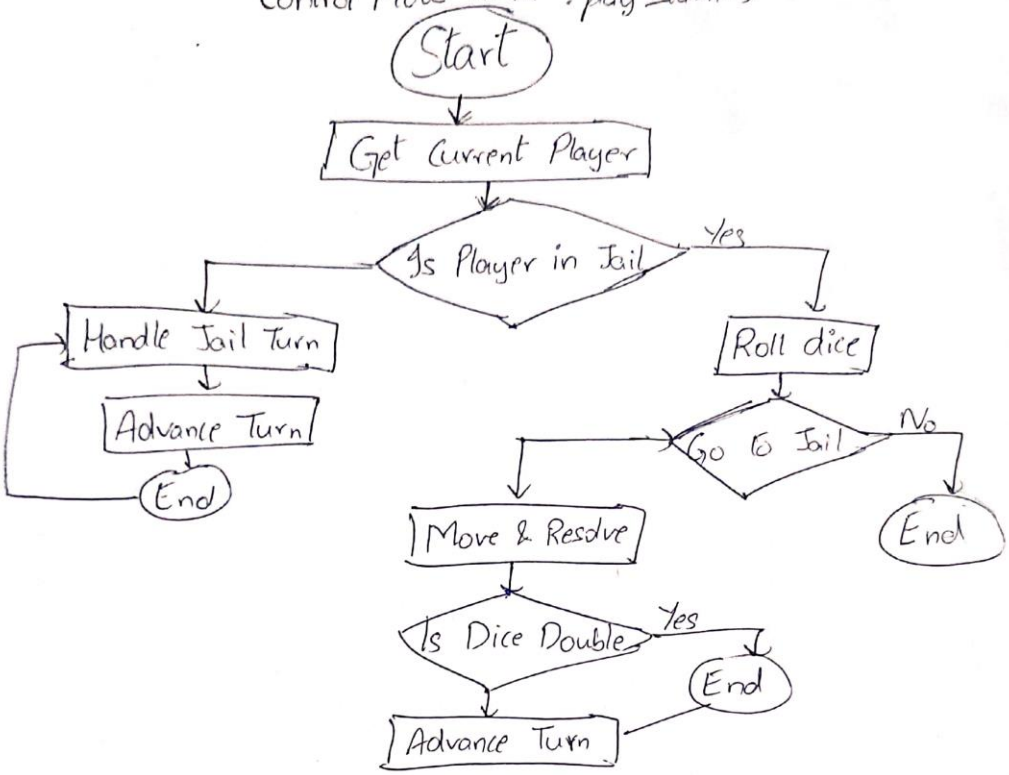
- Player in jail condition
- Three consecutive doubles condition
- Dice roll doubles condition

Therefore:

M = 3 + 1 = **4**

This indicates that at least **4 independent paths** exist in the function, and all must be covered during testing.

Control Flow Graph . play_turn()



1.2 Code Quality Analysis

The MoneyPoly codebase was analyzed using static code analysis (pylint). Multiple issues related to code quality, readability, and maintainability were identified and resolved iteratively.

Iteration 1: Basic Cleanup

Issues addressed:

- Unused imports (e.g., math, os, sys)
- Missing module-level docstrings
- Missing class docstrings
- Improper boolean comparisons (== True)
- Use of bare except blocks

Fixes applied:

- Removed unused imports
- Added descriptive module and class docstrings
- Replaced if x == True with if x
- Replaced except: with except Exception:

Iteration 2: Import Path Corrections

After restructuring the project, import statements referring to the moneypoly package became invalid.

Issues addressed:

- Broken imports such as from moneypoly.config import ...

Fixes applied:

- Updated all imports to use local module paths (e.g., from config import ...)
- Ensured all modules could be executed without import errors

---

Iteration 3: Validation and Logical Improvements

Issues addressed:

- Missing validation for player count
- Lack of input validation in financial operations
- Logical inconsistencies in core gameplay functions

Fixes applied:

- Added validation to ensure at least two players are required
- Added checks to prevent invalid monetary operations
- Improved robustness of critical functions

1.3 White Box Test Cases and Bug Fixes

White-box testing was performed using pytest to ensure that all internal paths, edge cases, and decision points were thoroughly tested.

Test Case Design

The following categories of test cases were designed:

1. Branch Coverage

- Jail vs non-jail execution paths
- Doubles vs non-doubles conditions
- Property vs special tile handling

---

2. Edge Cases

- Minimum number of players
- Negative monetary values
- Insufficient balance scenarios

---

3. Key State Testing

- Player balance updates
- Dice roll outcomes

- Property ownership conditions

Test Cases Implemented

Test Case 1: Minimum Player Validation

- Scenario: Game initialized with fewer than 2 players
- Expected Result: ValueError raised
- Purpose: Ensures valid game initialization

Test Case 2: Bank Negative Collection

- Scenario: Negative amount passed to bank collection
- Expected Result: Error raised
- Initial Issue: Negative values were accepted
- Fix: Added validation to restrict negative inputs

Test Case 3: Player Balance Deduction

- Scenario: Deducting more than available balance
- Expected Result: Error raised
- Initial Issue: Balance could become negative
- Fix: Added validation to prevent overdraft

Test Case 4: Dice Roll Range

- Scenario: Dice rolled multiple times
- Expected Result: Values between 2 and 12
- Initial Issue: Dice used incorrect range (1–5)
- Fix: Updated range to 1–6

Test Case 5: Winner Selection Logic

- Scenario: Player with highest balance should win
- Expected Result: Player with maximum net worth selected
- Initial Issue: Minimum value was selected instead
- Fix: Replaced min() with max()

Test Case 6: Property Group Ownership

- Scenario: Player owns all properties in a group
- Expected Result: Full ownership detected
- Initial Issue: Used any() instead of all()
- Fix: Corrected logic to ensure complete ownership

Bugs Identified and Fixes

Error 1: Bank Collection Validation

- Issue: Negative values accepted
- Fix: Added validation to ensure only positive values

Error 2: Player Balance Validation

- Issue: Player balance could become negative
- Fix: Added checks for sufficient balance

Error 3: Dice Value Range

- Issue: Dice values ranged from 1 to 5
- Fix: Corrected to 1 to 6

Error 4: Winner Selection Logic

- Issue: Incorrect player selected as winner
- Fix: Updated logic to select maximum net worth

Error 5: Property Group Ownership

- Issue: Incorrect ownership detection

- Fix: Ensured all properties must belong to the player

Conclusion

White-box testing enabled thorough analysis of the internal structure of the MoneyPoly system. By designing test cases that cover all branches and edge conditions, several logical and structural issues were identified and corrected.

The iterative approach using static analysis and unit testing significantly improved code quality, correctness, and robustness of the system.